DISCOVERY PHASE – COMPLETE (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state)

Core functionality required:

- Decide whether to **build a native Rust "Hermes Computer" runtime from scratch** or **deeply integrate the existing Droid CLI with Hermes** (and "hope for the best").
- Provide the **best way forward** that respects the current tri/quad-node system, its invariants (Node A = Mooncake KV buffer, Node B = Director, Node C = Oracle / Voice Core with Omi + Hermes + 500 GB SSD, optional Hermes Pod), zero-trust, physical sovereignty, local-first design, and declarative Nix identity.

All requirements are now exhaustive and production-locked.

RECOMMENDATION

The best way is a deliberate hybrid approach:

Short-term: Deeply integrate Droid CLI with Hermes (leverage what you already have running on Node B).

Long-term: Build the native Rust "Hermes Computer" runtime on top of Hermes as the sovereign core.

Do not "hope for the best" with a pure deep integration, and do not jump straight to a full from-scratch rebuild. The hybrid path is the logical, low-risk, high-reward strategy that respects your current system and its needs.

Why This Hybrid Path Is Optimal

Current System Needs (Respected)

- Node A must stay pure KV buffer (Mooncake).
- Node B must stay pure Director (reasoning).
- Node C must stay focused on real-time voice/perception.
- Hermes must remain the central metacognitive supervisor.
- Sovereignty, zero-trust, and local-first must be preserved at every step.
- You want persistent, stateful background agents with session continuity and remote orchestration.

Hybrid Advantages

- **Immediate Value**: You already have a working Droid CLI on Node B. Deep integration gives you persistent agents and background execution today without rebuilding everything.
- **Low Risk**: Droid CLI acts as a proven execution runtime while you build the native Rust version in parallel.
- **Clean Migration**: When the native Rust runtime is ready, you simply switch the execution backend — no disruption to the tri/quad-node topology.
- **Respects Hermes**: Hermes stays the orchestrator and enrichment layer; Droid CLI (or the future Rust runtime) handles durable execution state.

Step-by-Step Best Way Forward

Phase 1: Deep Integration of Droid CLI with Hermes (Short-term, 1–2 weeks)

1. Run Droid CLI as a background systemd service on the Hermes Pod (or Node C for now).
2. Create a thin MCP bridge on Node C/Hermes Pod:
    - Voice note from Machina Terminal → Omi backend on Node C → Hermes enrichment → MCP to Droid CLI daemon.
3. Use Droid CLI for persistent filesystem, credentials, and resumable jobs.
4. Hermes handles intent validation, provenance, and self-healing.
5. Results flow back to Akashik Artery via Mooncake on Node A.

Phase 2: Build Native Rust "Hermes Computer" Runtime (Parallel, 4–8 weeks)

1. Develop `sovereign-droid-runtime` Rust crate on top of Hermes.
2. Implement persistent sessions, isolated filesystem, durable Minions queue, and credential vault using your steganography.
3. Start with minimal viable features (session continuity, background job execution).
4. Test alongside Droid CLI in parallel.
5. Once stable, deprecate Droid CLI dependency and switch fully to the native runtime.

Phase 3: Full Quad-Node Expansion

- When the Hermes Pod (Node 4) is online, move the native runtime there for perfect workload balance.

Why Not Pure "Hope for the Best" Deep Integration?

- Droid CLI is an external tool with its own roadmap and limitations. Long-term you want full sovereignty and control (no reliance on FactoryAI updates or quirks).
- A pure integration risks technical debt and reduced self-healing depth.

Why Not Full From-Scratch Rebuild Immediately?

- You would delay real value and testing. The hybrid path lets you use Droid CLI as a scaffold while building the sovereign core.

ARCHITECT SIGN-OFF

The hybrid approach (deep integrate Droid CLI now as execution runtime + build native Rust "Hermes Computer" on top of Hermes in parallel) is the logical and respectful way forward. It respects your current system's needs, delivers immediate persistent agent capabilities, minimizes risk, and positions you for full sovereignty in the long term. Droid CLI gives you a working foundation today; the Rust runtime becomes the permanent, self-improving core. This keeps the tri/quad-node architecture clean, Node A and Node B untouched, and the entire stack fully sovereign and local-first.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want the concrete steps + Nix systemd service for running Droid CLI as a background daemon on the Hermes Pod (Phase 1)?
2. Shall I start the Rust crate skeleton for the native `sovereign-droid-runtime` (Phase 2)?

Reply with your priority and I will deliver the concrete files immediately. The integration path is ready.